

Optimisation in dMod2

Hessian sources, seeding, and the economics of a multi-start

Simon Beyer

2026-09-06

Contents

1	Introduction	2
2	The trust-region core	3
2.1	Termination	3
3	The source of the model Hessian	4
3.1	The hybrid switch	4
4	Seeding and memory of the quasi-Newton approximation	5
5	Multi-start behaviour	6
6	Discussion	8
7	Reproducing the numbers	8
	References	8

1 Introduction

Parameter estimation for ordinary differential equation models is a non-convex problem solved by local optimisation from many starting points. dMod2 [1] provides `trust()`, a trust-region method, and `mstrust()`, which runs it from many starts. This vignette describes their arguments and the behaviour those produce.

The example throughout is the STAT5 dimerisation model of Boehm et al. [2], built as in the shipped example `example_Boehm_JProteomeRes2014.R`: nine parameters on a base-10 logarithmic scale, a weak prior on the six dynamic ones, and bounds of ± 5 . Every table and figure below rests on one shared set of 100 starting points.

```
data(boehm)
mydataL <- as.datalist(boehm)

epo <- "1.25e-7*exp(-Epo_degradation_BaF3*time)"
reactions <- eqnlist() |>
  assignCompartment(nucpApA = "nuc", nucpApB = "nuc", nucpBpB = "nuc",
    volume = "0.45") |>
  addReaction("2*STAT5A", "pApA", paste0("k_phos*", epo, "*STAT5A^2"),
    compartment = "cyt") |>
  addReaction("STAT5A + STAT5B", "pApB", paste0("k_phos*", epo, "*STAT5A*STAT5B"),
    compartment = "cyt") |>
  addReaction("2*STAT5B", "pBpB", paste0("k_phos*", epo, "*STAT5B^2"),
    compartment = "cyt") |>
  addReaction("pApA", "nucpApA", "k_imp_homo*pApA", compartment = "cyt") |>
  addReaction("pApB", "nucpApB", "k_imp_hetero*pApB", compartment = "cyt") |>
  addReaction("pBpB", "nucpBpB", "k_imp_homo*pBpB", compartment = "cyt") |>
  addReaction("nucpApA", "2*STAT5A", "k_exp_homo*nucpApA", compartment = "nuc") |>
  addReaction("nucpApB", "STAT5A + STAT5B", "k_exp_hetero*nucpApB",
    compartment = "nuc") |>
  addReaction("nucpBpB", "2*STAT5B", "k_exp_homo*nucpBpB", compartment = "nuc") |>
  setCompartmentVolume(cyt = "1.4")

# The optimiser can only resolve what the integrator delivers, so the tolerances
# are set explicitly rather than left at their defaults.
myOptionsODE <- list(atol = 1e-8, rtol = 1e-6, maxattempts = 100L, maxsteps = 1e6)
model <- odemodel(reactions, modelname = "boehm_ode", compile = FALSE,
  outdir = .outdir)
x <- Xs(model, optionsOde = myOptionsODE, optionsSens = myOptionsODE)

observables <- eqnvec(
  pSTAT5A_rel = paste0("(100*pApB + 200*pApA*specC17)/",
    "(pApB + STAT5A*specC17 + 2*pApA*specC17)"),
  pSTAT5B_rel = paste0("-(100*pApB - 200*pBpB*(specC17 - 1))/",
    "((STAT5B*(specC17 - 1) - pApB) + 2*pBpB*(specC17 - 1))"),
  rSTAT5A_rel = paste0("(100*pApB + 100*STAT5A*specC17 + 200*pApA*specC17)/",
    "(2*pApB + STAT5A*specC17 + 2*pApA*specC17",
    " - STAT5B*(specC17 - 1) - 2*pBpB*(specC17 - 1))"))
errorModels <- eqnvec(pSTAT5A_rel = "sd_pSTAT5A_rel",
  pSTAT5B_rel = "sd_pSTAT5B_rel",
  rSTAT5A_rel = "sd_rSTAT5A_rel")

g <- Y(observables, x, modelname = "boehm_obs", attach.input = FALSE,
  compile = FALSE, outdir = .outdir)
e <- Y(errorModels, g, modelname = "boehm_err", attach.input = FALSE,
  compile = FALSE, outdir = .outdir)

innerpars <- getParameters(model, g, e)
estimated <- c("Epo_degradation_BaF3", "k_exp_hetero", "k_exp_homo",
  "k_imp_hetero", "k_imp_homo", "k_phos",
```

```

      "sd_pSTAT5A_rel", "sd_pSTAT5B_rel", "sd_rSTAT5A_rel")
p <- eqnvec() |>
  define("x-x", x = innerpars) |>
  define("x-0", x = c("pApA", "pApB", "pBpB", "nucpApA", "nucpApB", "nucpBpB")) |>
  insert("STAT5A ~ 207.6*ratio") |>
  insert("STAT5B ~ 207.6 - 207.6*ratio") |>
  insert("ratio ~ 0.693") |>
  insert("specC17 ~ 0.107") |>
  insert("x-exp10(x)", x = estimated) |>
P(modelname = "boehm_trafo", condition = "Boehm2014", compile = FALSE,
  outdir = .outdir)

compile(g, x, p, e, output = "boehm_vig", cores = 2)

prd      <- g*x*p
outerpars <- getParameters(prd)
pouter   <- structure(rep(-1, length(outerpars)), names = outerpars)
dyn       <- setdiff(outerpars, grep("^sd_", outerpars, value = TRUE))
obj       <- normL2(mydataL, prd, e) + constraintL2(pouter[dyn], sigma = 4)

parlower <- structure(rep(-5, length(outerpars)), names = outerpars)
parupper <- structure(rep( 5, length(outerpars)), names = outerpars)

length(pouter)
#> [1] 9

```

2 The trust-region core

`trust()` minimises a smooth objective by repeatedly solving the constrained quadratic subproblem

$$\min_{\|p\| \leq r} g^\top p + \frac{1}{2} p^\top H p,$$

exactly, through a full eigendecomposition of H [3]. Box constraints are handled by the interior scheme of Coleman and Li [4], in which the distance to the bound that the gradient pushes towards enters the metric of the subproblem. Iterates therefore remain strictly inside the feasible box and never land on its boundary; the `atBound` component of the result reports which parameters a bound is holding.

The subproblem is solved exactly rather than by truncated conjugate gradients because the quasi-Newton updates described below can produce an indefinite H : the Moré-Sorensen solver treats that case directly, whereas a Steihaug-type method can only stop at the trust-region boundary.

2.1 Termination

Six tests can end a run, and the `stopReason` component names the one that fired. Which of them apply depends on where the Hessian comes from.

stopReason	criterion	evaluated on
gradient	$\max v \cdot g \leq \text{gtol}$	the current iterate
fvalue	$ f - f_{\text{try}} < \text{ftol}$	the last accepted step
preddiff	predicted decrease $< \text{mtol}$	the last accepted step
step	step norm $< \text{xtol}$	the last accepted step
stagnation	five rejected steps leaving f flat	recent steps
radius	radius $< \text{rmin}$; reports <code>converged = FALSE</code>	the radius

Only the first tests first-order optimality. Three of the others, `fvalue`, `preddiff` and `step`, read the prediction of the quadratic model, so a Hessian that misrepresents the local curvature can trip them at a point that is not stationary. The hybrid switch below exploits that asymmetry; it also accounts for a fit reporting `converged = TRUE` at an implausibly high objective value.

Which tests may end a run follows from the same distinction. While the model is the objective's own $J^T J$, its prediction is a curvature statement and the three apply as given. During a quasi-Newton phase the model is an approximation still being built, so its verdict describes the approximation rather than the iterate; the three are then suppressed and the run ends on `gradient`, on `stagnation`, which counts rejected steps instead of reading the model, or on the radius. The gradient norm is the termination criterion usually recommended for quasi-Newton methods [5].

3 The source of the model Hessian

The argument `hessianMethod` selects where H comes from. The subproblem solver, the scaling and the bound handling are identical in all four cases.

value	Hessian	required from the objective
"gn"	the objective's $J^T J$, rebuilt each iteration	value, gradient, Hessian
"bfgs"	a Powell-damped BFGS update [6]	value, gradient
"sr1"	a symmetric rank-one update, possibly indefinite	value, gradient
"hybrid"	"gn", then "bfgs" after the first soft stop	value, gradient, Hessian

For a sum of squares the Gauss-Newton matrix $J^T J$ is an adequate curvature model wherever the residuals are small, and in `dMod2` it is obtained without additional cost: the same Jacobian yields both the gradient and the Hessian, so a gradient and a full derivative evaluation are measurably the same price. This is why "gn" is the default.

The Gauss-Newton matrix omits the term involving residuals times second derivatives. Close to the optimum the quadratic model therefore ceases to improve, and `ftol` or `mtol` fires while the iterate is still short of the minimum. A quasi-Newton approximation does not carry that bias, since it infers curvature from the steps actually taken.

3.1 The hybrid switch

"hybrid" runs "gn" until the first of `fvalue`, `preddiff`, `step` or `stagnation` would terminate the run, then switches once to "bfgs" and continues from the last accepted $J^T J$ as its seed. The `gradient` test remains terminal, since it is a genuine optimality statement rather than a symptom of a stalled model. The construction follows Fides [7], which switches when the trust radius stops updating. The `qnEval` component counts the objective evaluations spent after the switch.

```
set.seed(1L)
start <- as.parvec(msParframe(pouter, n = 1, sd = 3))

single <- .fit("single", list(
  gn      = trust(obj, start, rinit = 0.1, rmax = 10, iterlim = 5000,
    parlower = parlower, parupper = parupper),
  hybrid = trust(obj, start, rinit = 0.1, rmax = 10, iterlim = 5000,
    parlower = parlower, parupper = parupper,
    hessianMethod = "hybrid")))

data.frame(method = names(single),
```

```

value = vapply(single, `[`, 0.0, "value"),
stop = vapply(single, `[`, "", "stopReason"),
neval = vapply(single, `[`, 0L, "neval"),
qnEval = vapply(single, `[`, 0L, "qnEval"),
row.names = NULL)
#>  method      value      stop neval qnEval
#> 1    gn 297.3323      fvalue   105     0
#> 2 hybrid 297.3260 stagnation  143    38

```

Over the shared starting points of the next section the hybrid spends a few per cent more evaluations than "gn" and reaches a lower median value. The `grad` column of the table there counts the starts that end at a stationary point, which the hybrid reaches and "gn" does not.

4 Seeding and memory of the quasi-Newton approximation

Three arguments shape the approximation itself. They are read when the run begins in a quasi-Newton phase, that is for "bfgs" and "sr1"; "gn" ignores them, and "hybrid" seeds its handover from the Gauss-Newton matrix irrespective of `hessianInit`.

`hessianInit` selects the initial matrix. The default "gn" uses the objective's Hessian at `parinit`. The alternative "identity" starts from the identity and additionally stops `trust()` requesting a Hessian at all, which is what a derivative scheme returning only gradients would be able to supply.

The expectation that a genuine Hessian must be the better seed does not survive a multi-start, which is why the identity is the seed used throughout the rest of this vignette. At a starting point three decades away from the optimum, $J^T J$ is large and severely ill-conditioned. A BFGS update is of rank two and preserves positive definiteness, so it cannot cut such a seed down along the directions where it is wrong by orders of magnitude, and the approximation stays dominated by curvature that was never right; on this model those runs exhaust the iteration cap rather than reaching a termination test. "sr1" is not affected, because its update is of rank one and carries no sign constraint, so it can overwrite bad curvature outright, and the indefinite approximations that follow are what the Moré-Sorensen subproblem solves. `inst/benchmarks` measures both seeds side by side.

`qnMemory` bounds the curvature that is carried forward. The default 0 accumulates every update onto the seed, which is what the rest of this vignette uses. A positive value rebuilds the approximation in each iteration from γI together with the most recent `qnMemory` pairs, where γ is the Shanno-Phua scaling $y^T y / s^T y$ of the newest pair [8]. This is the curvature information of L-BFGS [5], but assembled into the explicit matrix the subproblem requires, so it saves no memory. It is meant for problems where carrying a dense $n_\theta \times n_\theta$ matrix is itself the binding cost, which begins at parameter counts an order of magnitude above anything shown here.

`qnCautious` rejects a BFGS pair whose curvature along the step is negligible, $s^T y \leq \text{qnCautious} \cdot \|s\| \|y\|$ [9]; `qnSkipped` counts the rejections. On this model it changes neither the outcome nor the cost.

```

seeds <- .fit("seeds", lapply(list(
  list(hessianMethod = "bfgs"),
  list(hessianMethod = "bfgs", hessianInit = "identity"),
  list(hessianMethod = "bfgs", hessianInit = "identity", qnMemory = 10L)),
function(a) do.call(trust, c(list(obj, start, rinit = 0.1, rmax = 10,
                                iterlim = 5000, parlower = parlower,
                                parupper = parupper), a))))

data.frame(seed = c("gn", "identity", "identity"),
  memory = c(0L, 0L, 10L),
  value = vapply(seeds, `[`, 0.0, "value"),
  stop = vapply(seeds, `[`, "", "stopReason"),
  neval = vapply(seeds, `[`, 0L, "neval"))
#>      seed memory      value      stop neval

```

```
#> 1      gn      0 297.3260 stagnation 107
#> 2 identity 0 279.6607 stagnation 139
#> 3 identity 10 499.4909 stagnation 84
```

5 Multi-start behaviour

`mstrust()` runs `trust()` from many starting points. Drawing them once with `msParframe()` and passing the resulting frame as `center` ensures that a comparison between methods differs only in the method.

```
set.seed(20260905)
starts <- msParframe(pouter, n = 100, sd = 3)

fitAll <- function(...)
  as.parframe(mstrust(obj, center = starts, fits = nrow(starts),
    cores = .cores, rinit = 0.1, rmax = 10, iterlim = 5000,
    parlower = parlower, parupper = parupper, ...))

settings <- list(
  `gn`      = list(hessianMethod = "gn"),
  `hybrid`  = list(hessianMethod = "hybrid"),
  `bfgs, id` = list(hessianMethod = "bfgs", hessianInit = "identity"),
  `sr1, id` = list(hessianMethod = "sr1", hessianInit = "identity"))

runs <- .fit("multistart", lapply(settings, function(a) do.call(fitAll, a)))
```

A start counts as a hit when it lands within 0.1 of the best value seen anywhere.

```
best <- min(vapply(runs, function(f) min(f$value), 0.0))

summary <- function(f)
  data.frame(conv = sum(f$converged),
    grad = sum(f$stopReason == "gradient"),
    hits = sum(f$value <= best + 0.1),
    neval = sum(f$neval),
    medEval = stats::median(f$neval),
    medValue = stats::median(f$value))

cbind(source = names(runs), do.call(rbind, lapply(runs, summary)),
  row.names = NULL)
#>      source conv grad hits neval medEval medValue
#> 1      gn 100   0   6 35536 219.0 297.3301
#> 2  hybrid 100  13   6 38517 244.5 297.3260
#> 3 bfgs, id 100  30  13 11162 120.5 301.2019
#> 4 sr1, id 100  22  18 17510 186.5 299.2640
```

The waterfall plot sorts the final objective values in ascending order. A flat leading plateau indicates repeated arrival at the same optimum, and the height of the first step how well that optimum is separated from its competitors. `plotValues()` draws one collection, shaded by iteration count; the figure below ranks the four separately and colours them by method. A circle marks a converged fit, a triangle one that stopped on any other test.

```
# One panel for several fit collections, each ranked on its own. Drawn back to
# front, so the first one listed is not buried, and without the palette's
# pale yellow.
waterfall <- function(frames, cutoff = Inf) {
  ranked <- do.call(rbind, lapply(rev(names(frames)), function(set) {
    fits <- subset(frames[[set]], value < cutoff)
    fits <- fits[order(fits$value), ]
```

```

    data.frame(index = seq_len(nrow(fits)), value = fits$value,
               converged = as.character(fits$converged), set = set)
  )))
ranked$set <- factor(ranked$set, levels = names(frames))
colours <- stats::setNames(dMod_palette(8, "okabe")[-5][seq_along(frames)],
                          names(frames))

ggplot(ranked, aes(index, value, colour = set)) +
  geom_line(aes(group = set), linewidth = 0.3) +
  geom_point(aes(shape = converged), size = 1.3) +
  scale_shape_manual(values = c(`TRUE` = 16, `FALSE` = 17),
                    limits = c("TRUE", "FALSE"), labels = c("yes", "no")) +
  scale_colour_manual(values = colours) +
  labs(x = "index", y = "value", colour = NULL) +
  theme_dMod() +
  theme(legend.position = "bottom", legend.box = "vertical")
}

waterfall(runs, cutoff = 600)

```

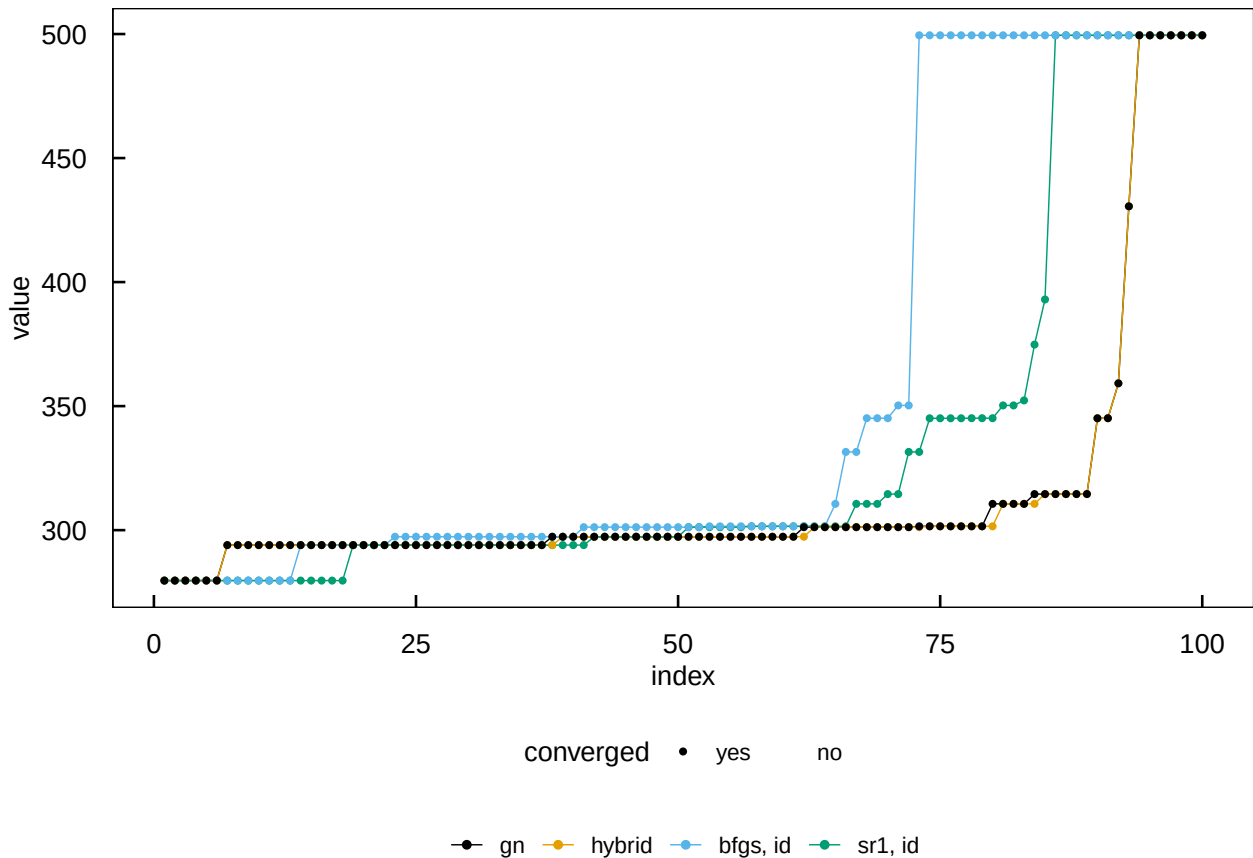


Figure 1: Waterfall plots of the four Hessian sources over the same 100 starting points, each ranked separately. Values above 600 are omitted.

The curves share their starting points and therefore their basins, and they fall into two shapes. Gauss-Newton and the hybrid track each other over the whole range, as they must, since the hybrid is Gauss-Newton until a soft stop fires; they differ in the leading plateau, which is not flat under Gauss-Newton because the model stops improving before the minimum is reached, whereas the quasi-Newton phase carries those starts onto a common value. The two identity-seeded quasi-Newton runs have a different shape altogether: they reach the

global optimum from more starting points and abandon more of them on the poor plateau, so they explore more widely and end on a worse typical value.

The `stopReason` of the runs shows the mechanism. Gauss-Newton ends on the model tests, which for it are legitimate. The quasi-Newton runs, which those tests no longer terminate, end on `stagnation` or on `gradient`.

6 Discussion

The four sources differ in what a single start buys. "gn" and the hybrid produce the better typical run, reaching the lower median value, and the hybrid additionally resolves the leading plateau that "gn" leaves ragged, for a few per cent more evaluations. The two identity-seeded quasi-Newton sources spend fewer evaluations on a start and reach the published optimum from more of them, at a worse typical value. Which of those matters depends on whether starting points or evaluations are the scarce resource.

`parscale`, the diagonal preconditioner, is off by default and stays off here. It has nothing to correct when parameters are already on a logarithmic scale, which is the normal case in `dMod2`, and the ill-conditioning of sloppy models lies along rotated directions [10] that no diagonal scaling reaches.

The measurements come from a single model with nine estimated parameters, and the advantage of a quasi-Newton method is expected to narrow as the parameter count grows, since a dense update requires of the order of n_θ accepted steps before it carries curvature along every direction. The hit counts are single digit to low double digit out of 100 starts and carry a corresponding sampling error, so they do not establish an ordering between two sources that lie close together. A ranking that carried weight would have to run over a set of models.

7 Reproducing the numbers

The fits behind the tables and figures are stored in `dev/optimisation/results.rds` and reused, so this document renders in seconds and reproduces exactly what it shows; `dev/render-optimisation.R --refit` recomputes them. The script `bench_hessianSource.R` under `inst/benchmarks` runs the same comparison outside the vignette, over both seeds and with the per-start `stopReason` tables.

References

- [1] D. Kaschek, W. Mader, M. Fehling-Kaschek, M. Rosenblatt, and J. Timmer, "dMod: Efficient parameter estimation and uncertainty analysis for dynamic models," *Journal of Statistical Software*, vol. 88, no. 10, pp. 1–32, 2019, doi: 10.18637/jss.v088.i10.
- [2] M. E. Boehm, L. Adlung, M. Schilling, S. Roth, U. Klingmüller, and W. D. Lehmann, "Identification of isoform-specific dynamics in phosphorylation-dependent STAT5 dimerization by quantitative mass spectrometry and mathematical modeling," *Journal of Proteome Research*, vol. 13, no. 12, pp. 5685–5694, 2014, doi: 10.1021/pr5006923.
- [3] J. J. Moré and D. C. Sorensen, "Computing a trust region step," *SIAM Journal on Scientific and Statistical Computing*, vol. 4, no. 3, pp. 553–572, 1983, doi: 10.1137/0904038.
- [4] T. F. Coleman and Y. Li, "An interior trust region approach for nonlinear minimization subject to bounds," *SIAM Journal on Optimization*, vol. 6, no. 2, pp. 418–445, 1996, doi: 10.1137/0806023.
- [5] J. Nocedal and S. J. Wright, *Numerical optimization*, 2nd ed. Springer, 2006. doi: 10.1007/978-0-387-40065-5.
- [6] M. J. D. Powell, "A fast algorithm for nonlinearly constrained optimization calculations," *Lecture Notes in Mathematics*, vol. 630, pp. 144–157, 1978, doi: 10.1007/BFb0067703.
- [7] F. Fröhlich and P. K. Sorger, "Fides: Reliable trust-region optimization for parameter estimation of ODE models," *PLOS Computational Biology*, vol. 18, no. 7, p. e1010322, 2022, doi: 10.1371/journal.pcbi.1010322.

- [8] D. F. Shanno and K.-H. Phua, “Matrix conditioning and nonlinear optimization,” *Mathematical Programming*, vol. 14, no. 1, pp. 149–160, 1978, doi: 10.1007/BF01588962.
- [9] D.-H. Li and M. Fukushima, “A modified BFGS method and its global convergence in nonconvex minimization,” *Journal of Computational and Applied Mathematics*, vol. 129, no. 1–2, pp. 15–35, 2001, doi: 10.1016/S0377-0427(00)00540-9.
- [10] R. N. Gutenkunst, J. J. Waterfall, F. P. Casey, K. S. Brown, C. R. Myers, and J. P. Sethna, “Universally sloppy parameter sensitivities in systems biology models,” *PLOS Computational Biology*, vol. 3, no. 10, p. e189, 2007, doi: 10.1371/journal.pcbi.0030189.